MILESTONE 5

PRODUCT RAG

Evaluation Setup:

1. We have **embedded a set of products which were not used to tune hyperparameters** in milestone 4. Also, the queries that will be used to evaluate the search results are new.
2. The hyperparameters tuned in milestone 4 are listed below.
    a. **Temperature**: LLM's creativity
    b. **Top-K**: Before reranking and choosing best 3 how many products should be fetched from product catalog
    c. **Search Query Refiner prompt**: Search query given by the user may not be complete. With the context available LLM will refine the search query
    d. **Reranker Prompt**: The results we get from product RAG follows cosine similarity and it need not be correct. So, an LLM performs reranking on a large set and suggest a small set of well curated products
3. The product info available in the product catalog had unwanted info, like index_code, section_no, etc which are codes for identifying the product. These **unwanted columns were removed before embedding**.
4. After embedding, we applied **normalisation on product embedding** which facilitates faster vector search.
5. The test dataset contains 10 queries with both customer context and chat history associated with it. Please refer to this sheet to see the test queries.
6. We chose only 10 queries because of manual scoring constraint and throttling limit of Gemini free API restrictions.

Evaluation Metrics:

1. We have created **10 queries with various context and history.**
2. The results from these **10 queries will be manually evaluated and scored**. While scoring we are taking into account factors mentioned below.

a. During manual evaluation we **will check whether any filter mentioned by the user is reflected in the result**. (Eg: Customer asked blue jean, and whether the results contains blue jeans or not)
b. Products is recommended **based on gender**
c. **History is used or not** (Eg: In one of the prompts the user asks ‘what about red one?’ Here history needs to be used to find the product type for which the user is searching)
d. Fashion sense (Eg: Some prompts are open ended, the user does not specify the color or category of the product he/she needs. The query will be like ‘Need a matching pant for my red formal shirt’. Here **the model needs to think and come up with a proper product category** and search for the same in product catalog)

Hyperparameter combinations:

Combination 1:

```
Temp: 0.2
Top-K: 5
Refine Prompt: refine_prompt_contextual
Rerank Prompt: rerank_prompt_contextual
```

Combination 2:

```
Temp: 0
Top-K: 10
Refine Prompt: refine_prompt_precise
Rerank Prompt: rerank_prompt_precise
```

Combination 3:

```
Temp: 0.5
Top-K: 12
Refine Prompt: refine_prompt_balanced
Rerank Prompt: rerank_prompt_balanced
```

Combination 4:

```
Temp: 0.8
Top-K: 20
Refine Prompt: refine_prompt_creative
Rerank Prompt: rerank_prompt_creative
```

Quantitative Evaluation:

1. Upon evaluating multiple combinations of hyperparameters mentioned above in milestone 4, the results were most promising when **combination-3's** hyperparameter setting was used.
    a. Temp: **0.5**
    b. Top-K: **12**
    c. Refine Prompt: **refine_prompt_balanced**
    d. Rerank Prompt: **rerank_prompt_balanced**

    Please refer to this link to find out the different prompts used. The prompt that performs better '**balanced set'**, is a balance between creativity and precise search and reranking. The LLM was asked to give importance in the following distribution

- 80% weight → user's **current query meaning**.
- 15% weight → **customer context** (age, gender, size, city, etc.).
- 5% weight → **past purchases**, to ensure consistent style or preference.

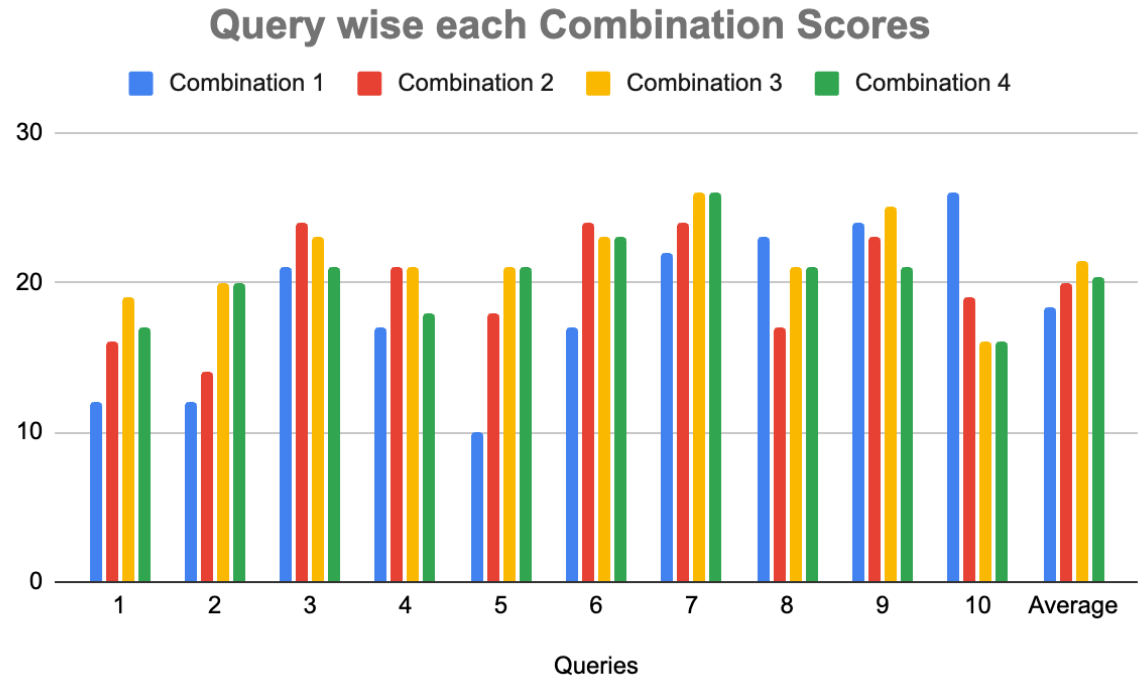
2. The same test is performed on the set of products which were not used while hyperparameter tuning. The scores obtained using different hyperparameter combinations are shown below.
3. The score we got on the validation set is 8.13 out of 10. This is an average of all the product suggestions shown by the chatbot.

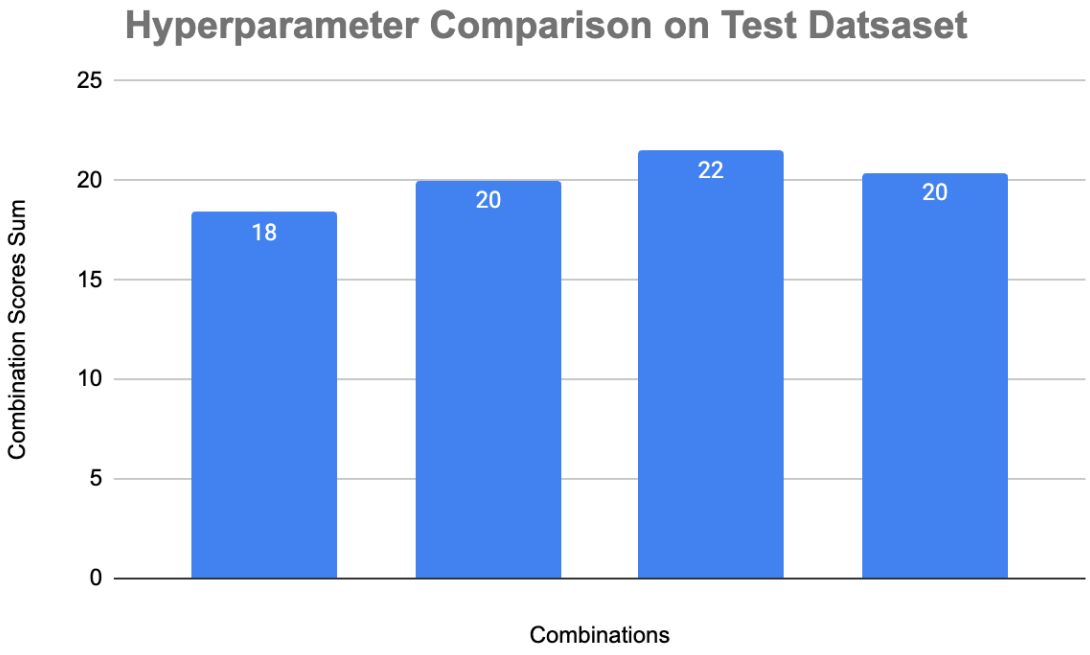
Hyperparameter combinations on Test Dataset:

Please refer to this sheet for more fine grained information.

Please refer to this log_file for results of each queries.

Please refer to this sheet for combination 3 - best hyperparameters combination product results





Combination 1 (Temp: 0.2 | Top-K: 5)

- Low temperature and narrow Top-K restricted the model's exploratory power.
- Produced repetitive, "safe" results lacking personalization.
- Often failed to capture contextual or stylistic nuances (e.g., fashion pairing or prior query reference).
- Result: Predictable but bland — suitable only for deterministic, rule-based tasks.

Combination 2 (Temp: 0 | Top-K: 10)

- Zero temperature made the model fully deterministic, improving filter accuracy.
- However, absence of stochasticity limited adaptability to ambiguous or stylistic requests.
- Worked well when user intent was explicit, but struggled with creative reasoning.
- Result: Reliable for exact filters; less effective for human-like conversational depth.

Combination 3 (Temp: 0.5 | Top-K: 12)

- Struck the perfect balance between stability and creativity.
- Moderate temperature enabled controlled randomness — allowing the model to explore multiple valid interpretations while maintaining coherence.
- Top-K of 12 ensured enough lexical diversity to handle follow-up or context-based queries.
- Delivered the highest average score (21.5), excelling in both context understanding and stylistic intelligence.
- **Result: Best configuration** - deal for production use; blends precision, adaptability, and natural conversation flow.

Combination 4 (Temp: 0.8 | Top-K: 20)

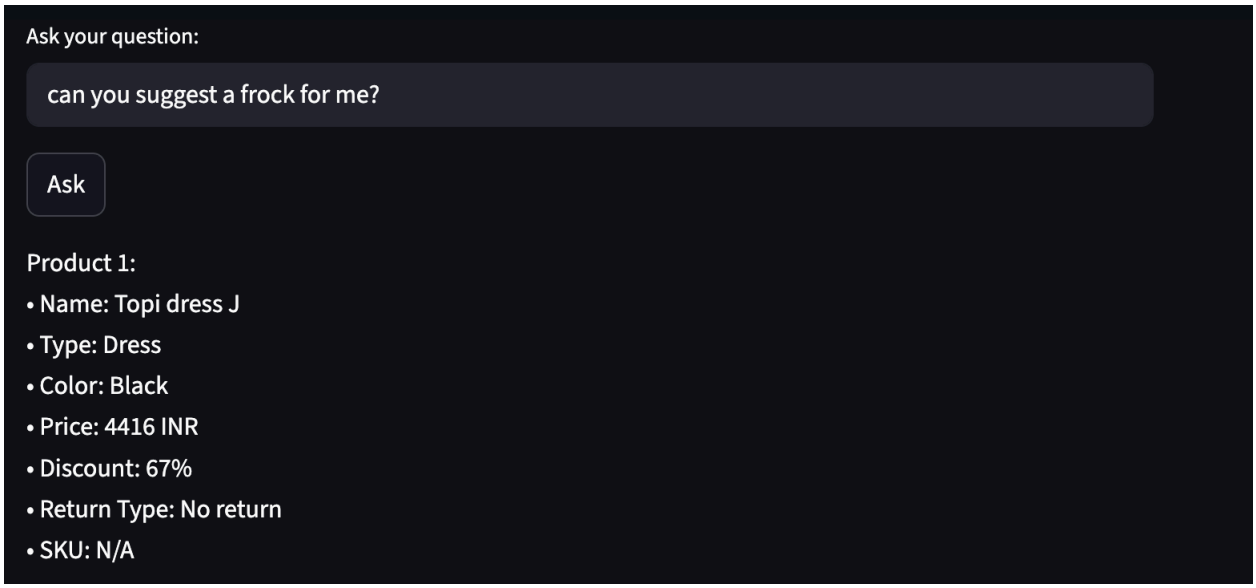
- High temperature enhanced creative diversity and fashion reasoning.
- However, greater randomness sometimes caused the model to drift from user filters or gender constraints.

- Strong performance in open-ended prompts but inconsistent under strict filtering scenarios.
- **Result:** Great for brainstorming or idea generation, less stable for precise recommendation use cases.

Conclusion: Combination 3 (Temp = 0.5, Top-K = 12) consistently outperformed others — offering the **best trade-off** between consistency, creativity, and contextual intelligence.
It captured fashion semantics effectively while maintaining alignment with user constraints, making it the most suitable for **real-world conversational recommendation systems**.

Qualitative Analysis:

1. The **chat bot can be instructed to ask more follow up questions**. Now it asks for basic info like age, gender. But here in the example shown below, the customer is a male and asks for a frock. So, in this case the bot should have said invalid product type and should have asked for proper info to proceed further



2. Currently the chatbot results in three most relevant products. Deliberately for one query we had only one relevant product in that case the other two products resulted were not matching the query's needs. **'need cream color jean' query resulted in one correct product and two wrong colored jeans.**

3. When **not much context is available** for the customer and nothing much is specified in the query, no prior transaction is made by the customer, **the results shown are very random** and suggestions can be improved.
4. Before applying the reranking chain, most of the search results have some irrelevant products which vary from the customer request. Upon applying the reranking chain, most of the irrelevant products get pushed down the list and products which match closely with the customer query are shown.
5. On common product categories the RAG works well and an example result is shown below.

   Query: Need a blue jean

   Result: **Refer next page**

Product 1:

- Name: Slim 5pkt
- Type: Trousers
- Color: Blue
- Price: 3563 INR
- Discount: 33%
- Return Type: 7 days return
- SKU: 449897049

Product 2:

- Name: Slim 5pkt
- Type: Trousers
- Color: Blue
- Price: 4191 INR
- Discount: 41%
- Return Type: 15 days return
- SKU: 449897018

Product 3:

- Name: 360 Flex Slim
- Type: Trousers
- Color: Blue
- Price: 2309 INR
- Discount: 42%
- Return Type: No return
- SKU: 540395003

Error Analysis:

1. When multiple filters are applied, for example: blue shirt, which is returnable, the results we get from vector search contains non returnable products also.

   Query: need a returnable blue t-shirt

   Results:

   As we can see in the results, products with return type - no return is also listed.

	prod_name	product_type_name	product_group_name	colour_group_name	department_name	Price_INR	Discount_Percentage	Return_Type
1697	Gary Tee	T-shirt	Garment Upper body	White	Jersey Fancy	2398	0	15 days return
1763	TD Drakeford gingham	Shirt	Garment Upper body	Dark Blue	Shirt	1510	62	15 days return
2045	Sam ls bd detailed	Shirt	Garment Upper body	Light Blue	Shirt	4646	40	7 days return
391	FRAME Easy Iron	Shirt	Garment Upper body	Dark Blue	Shirt S&T	4129	48	15 days return
387	FRAME Easy Iron	Shirt	Garment Upper body	Blue	Shirt S&T	4730	17	No return
489	3 PK V-NECK SS BASIC	T-shirt	Garment Upper body	Dark Blue	Jersey Basic	1300	64	7 days return
1717	Mike tee	T-shirt	Garment Upper body	Dark Blue	Jersey Fancy	3130	1	15 days return
1074	ROY SLIM RN T-SHIRT	T-shirt	Garment Upper body	White	Light Basic Jersey	774	52	7 days return
900	Pete bd ls poplin	Shirt	Garment Upper body	Blue	Shirt	4767	54	No return
138	CA Gustavsberg TVP	Shirt	Garment Upper body	Black	Shirt S&T	1985	64	No return
901	Pete bd ls poplin	Shirt	Garment Upper body	Dark Blue	Shirt	2765	5	No return
880	FRAME SS	Shirt	Garment Upper body	Dark Blue	Shirt	766	46	No return
2950	PRICE TEE ISW 21	T-shirt	Garment Upper body	Greenish Khaki	Tops Fancy Jersey	2603	46	7 days return
349	Boulevard tee	T-shirt	Garment Upper body	White	Tops Fancy Jersey	1971	37	No return
3263	Wow printed tee 6.99	T-shirt	Garment Upper body	Dark Blue	Jersey Fancy	2571	68	7 days return
1426	CAGE SLIM EASY IRON US	Shirt	Garment Upper body	White	Shirt S&T	1922	49	7 days return
1066	REX SLIM LS T-SHIRT	T-shirt	Garment Upper body	Black	Light Basic Jersey	3770	53	No return
1076	R-NECK SS SLIM FIT	T-shirt	Garment Upper body	Dark Blue	Light Basic Jersey	3412	51	No return
996	LARRY FANCY	Sweater	Garment Upper body	Dark Blue	Jersey Fancy	3537	43	15 days return
2165	Retro Slim	Trousers	Garment Lower body	Blue	Denim trousers	2189	7	7 days return

2. There is a **confusion between categories like shirt and t-shirt**. The embeddings for shirts and t-shirts seem to be close and results for t-shirts contain a large quantity of shirts and vice versa. This can be observed in the previous results image.

Limitations:

1. The current implementation of the system does not allow for **omission of already purchased or disliked items.**
2. When the chat grows big, the history in turn also increases in tokens. Having all of those in context increases the **latency of LLM calls.**
3. There is no way **users could upload images and search based on the images**. For example, similar clothes, or matching clothes for the uploaded image.
4. Using text embeddings alone, **matching visual pattern clothes cannot be found**. Only matching colors can be comprehended using text embeddings.

Next Steps:

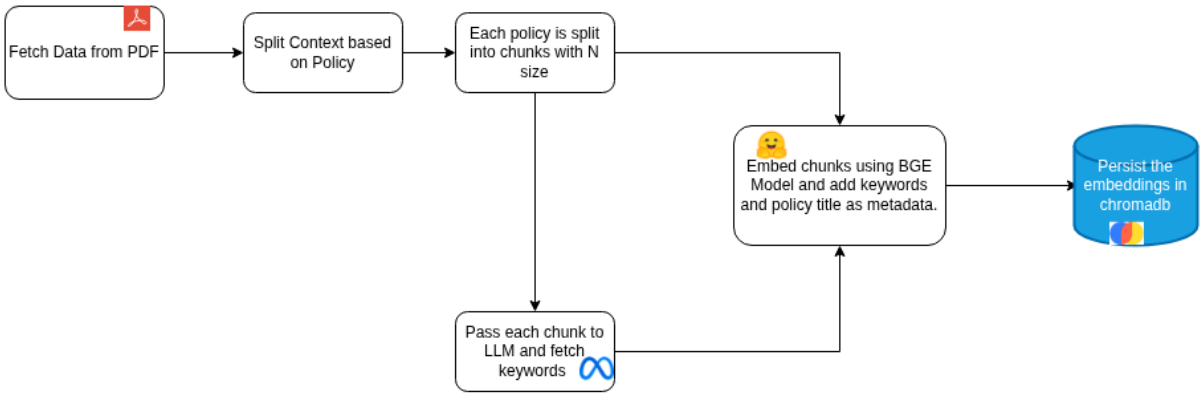
1. **Including image data into embedding** might be beneficial. The embedding quality will increase.
2. For accurate search results along with embedding search, **meta data search** using some deterministic method can be coupled.
3. To increase the embedding quality, embedding models can be **fine tuned using various images and their text descriptors**. Also, to make the embedding model to understand more about matching pairs, we could **train the embedding model to place matching clothes closer and other non matching pairs farther**. This could be achieved using loss functions like **triplet loss**.
4. For managing history, **other memory and token optimized techniques can be used**. (Eg: Summarizing history when the number of tokens increase beyond a threshold)
5. When the chatbot does not have any context about the customer, suggestions can be based on the l**ocation of the user and any other data we get from the http headers** or any other meta data.
6. A **customer classification model can be trained** to show suggestions based on purchases of other customers who fall in the same classification.

Policy RAG

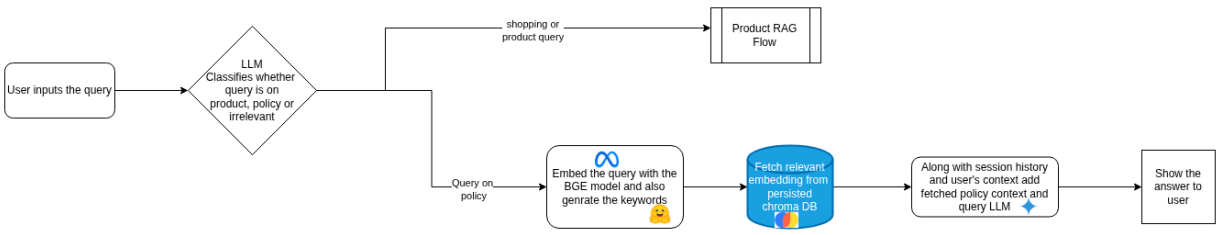
In Milestone 4
Following parameters chosen:

1. **Model**: BGE 1.5, allMini
2. **Chunk** size: 500,700
3. **Overlap**: 50,70
4. **Retrieval strategy:** Hybrid querying strategy and summarizing the output for setting LLM's context

Based on the results it was concluded that there needs to be an upgrade in Policy rag and it needs to be context aware.
Following is the updated Chunking strategy.



Following is the updated Retrieval Strategy:



Evaluation methodology:

RAGAS : Retrieval-**A**ugmented **G**eneration **As**sessment

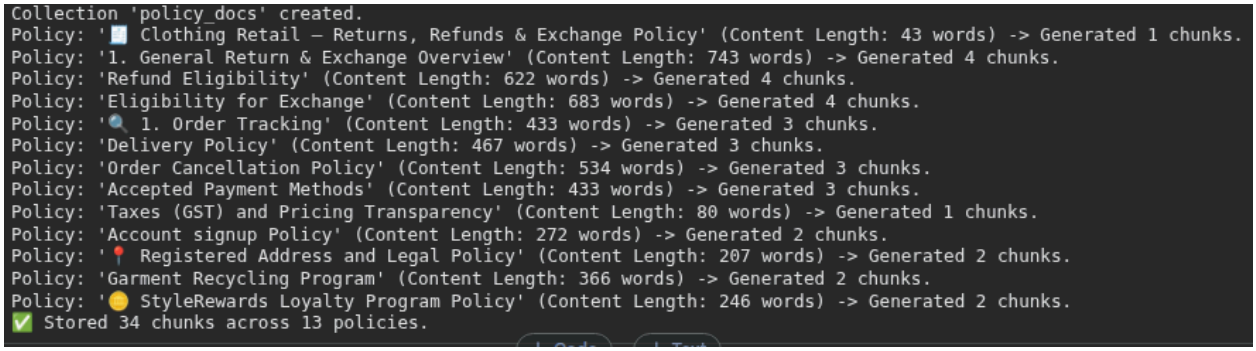
Based on the policy document, we formulated questions directly linked to the facts it outlined. Several questions drew from multiple policies, addressing areas such as the accrual and validity of loyalty points, as well as refund eligibility, including considerations related to GST and shipping charges.

Following Setup was used to perform Ragas evaluation:
Question: The user query provided as input to the RAG pipeline.
LLM Answer: The response generated by the RAG pipeline.
Contexts: The retrieved passages or documents from the external knowledge source that support the answer.
Ground Truths: The correct or reference answer used to assess the pipeline's performance.

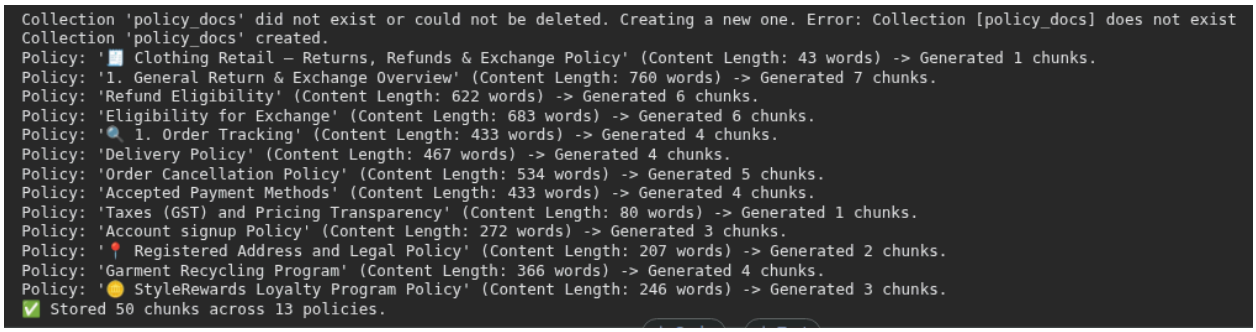
Outcome:

The average policy length is 395 words, so we tried with chunk sizes of 50, 100, 200.

Following depicts chunking with size 200 words



Following depicts chunking with size 120 words



Chunk sizes above 100 words proved ideal, as they provided sufficient context within each chunk. This improved retrieval quality and allowed the LLM to generate more accurate responses when combining retrieved context with user-specific information and prior chat history.

Pls check the below for performance of the new RAG system

ragas_evaluation_data.xlsx